Reporte de Avance: AgentIA v2 — Semana 12 a 18 de octubre de 2026

Gabriel Janampa Guzmán

19 de octubre de 2026

1. Qué se construyó

Esta semana se reescribió una versión anterior del sistema cuya lógica estaba codificada de forma fija, lo que dificultaba su mantenimiento. La refactorización separó responsabilidades, estandarizó la comunicación entre componentes y volvió configurables los pasos del flujo.

AgentIA v2 es el resultado: una plataforma que genera, verifica y repara microservicios Quarkus 3 / Java 21 de forma automática. El desarrollador describe el servicio en un archivo de especificación (*Blueprint*) y el sistema produce el código, lo compila, ejecuta las pruebas y, si algo falla, intenta corregirlo sin intervención manual.

Parte	**Qué hace**
Contratos	Define en Pydantic la forma exacta del Blueprint y de los mensajes entre componentes.
Orquestador LangGraph	Coordina los pasos (validar, generar, verificar, reparar) y guarda el estado de cada ejecución.
Microservicios FastAPI	Cada agente (validador, generador, verificador, reparador) corre como servicio HTTP independiente.
Agentes de dominio	Generan el código Quarkus (entidades JPA, mensajería Kafka, trazas OpenTelemetry) a partir de plantillas Jinja2.
Interfaz React	Permite ingresar el requerimiento, revisar y aprobar el Blueprint, ver el grafo de ejecución en vivo y elegir el modelo LLM.

Cuadro 1: Componentes principales de AgentIA v2

2. Cómo funciona el bucle principal

El proceso inicia con una descripción en lenguaje natural del requerimiento de negocio. El agente extractor (`SpecDraftingAgent`) la convierte en un *Blueprint* estructurado (historias de usuario BDD, modelos de dominio, base de datos y mensajería). El usuario revisa y aprueba la estructura de ese Blueprint antes de invocar a los agentes de código (puede devolverlo con ajustes). Con el contrato aprobado, el orquestador valida el esquema, genera el código Quarkus y lo verifica con Maven (compilación y pruebas). Si la verificación falla, un ciclo de reparación aplica parches acotados y vuelve a verificar, hasta un máximo de intentos configurable. El despliegue final (línea punteada) está planificado y aún no se ejecuta.

3. Resultados obtenidos

Durante las pruebas se registraron **33 ejecuciones** en la base de datos de control (`runs.db`) sobre diversos microservicios de negocio (`IniciacionPagosService`, `LiquidacionNetaDiferida`, `EscrowService`, entre otros): 13 culminaron con éxito (12 en su primer intento en $\sim$13 s y 1 tras una ronda de reparación) y 19 alcanzaron el límite de reintentos (`NO_FIX`) ante escenarios complejos

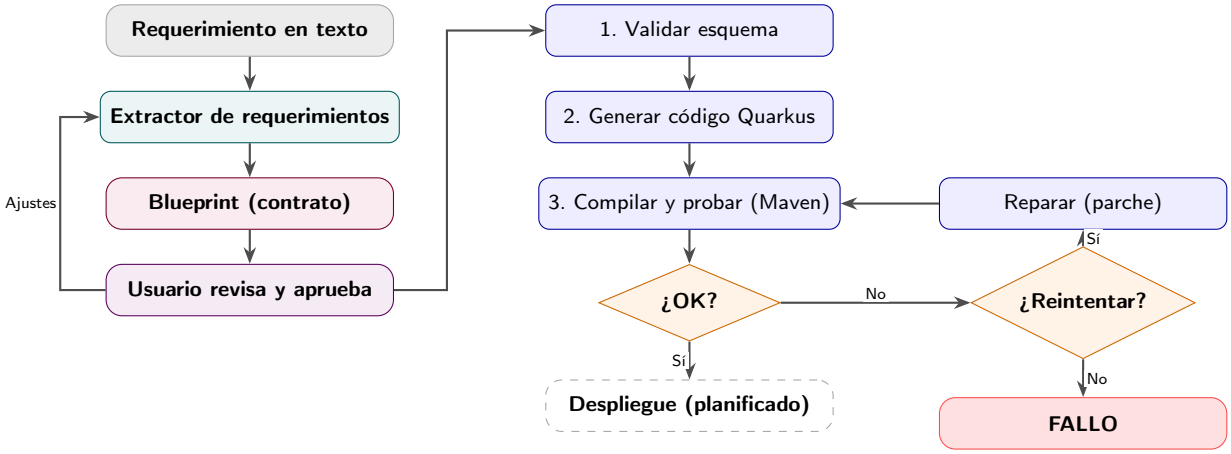


Figura 1: Flujo integral en dos fases: especificación/aprobación (izquierda) y bucle de ejecución/reparación (derecha)

(concurrencia, colas Kafka y transaccionalidad eventual). Esto confirma la solidez de la generación base y evidencia la necesidad de dotar al bucle de reparación de mayor contexto y memoria histórica.

4. Plan de trabajo a seguir (semana 19 a 25 de octubre de 2026)

El foco es hacer el bucle más inteligente y reproducible. Objetivos:

1. **Memoria de los agentes.** Hoy la memoria vive en el estado del grafo (`OrchestrationState`) y se guarda como JSON al terminar la traza. Se formalizarán dos niveles: *corto plazo* (historial de acciones y errores entre nodos, para no repetir reparaciones fallidas) y *largo plazo* (almacén persistente de soluciones previas que el agente consulta al generar o reparar nuevos servicios).

2. **Método de trabajo pstack.** (a) *Desarrollo:* se adoptarán las *skills* de pstack en el agente de codificación que desarrolla AgentIA: cada tarea pasa por una *skill* de diseño antes de escribir código y las decisiones quedan registradas. (b) *Aplicación en AgentIA:* se estudiará la metodología de pstack y se elegirán de tres a cuatro principios que encajen con el flujo actual, donde el usuario aprueba el Blueprint antes de los agentes de código (por ejemplo, aclarar ambigüedades del requerimiento con el usuario antes de redactar el Blueprint). Cada principio se medirá comparando intentos de reparación antes y después.

3. **Base de habilidades de generación con trazabilidad.** Un catálogo de plantillas reutilizables para patrones comunes (Outbox, saga, API REST paginada), cada una con sus pruebas de aceptación. El generador lo consultará antes de producir código desde cero, y cada habilidad aplicada quedará registrada en el estado de la traza para auditar qué produjo cada archivo.

4. **Validar el despliegue con Openship.** Openship es una plataforma de despliegue de código abierto (Apache 2.0) con CI/CD integrado. Es multiplataforma y autoalojable: permite pruebas locales ligeras o despliegue completo en un servidor con Docker (modo recomendado en Linux). Se usará para comprobar que un microservicio generado empaqueta su imagen y responde a su verificación de salud (`/q/health`), sin incurrir en costos de nube durante el desarrollo.